Unidade - 05

Arranjos de Dados: Matrizes, Vetores e etc...

Como definir um vetor?

```
REAL(KIND=8)::  a(5)
```

Como definir um vetor?

```
REAL(KIND=8):: a(5)
```

O resultado dessa declaração será a criação de cinco espaços de memória denotados por:

Como definir um vetor?

```
REAL(KIND=8):: a(5)
```

O resultado dessa declaração será a criação de cinco espaços de memória denotados por:

```
a(1) a(2) a(3) a(4) a(5)
```

Como definir um vetor?

```
REAL(KIND=8):: a(5)
```

O resultado dessa declaração será a criação de cinco espaços de memória denotados por:

```
a(1) a(2) a(3) a(4) a(5)
```

Que poderão ser tratados como cinco variáveis independentes.

Como definir um vetor?

```
REAL(KIND=8):: a(5)
```

O resultado dessa declaração será a criação de cinco espaços de memória denotados por:

```
a(1) a(2) a(3) a(4) a(5)
```

Que poderão ser tratados como cinco variáveis independentes.

OBS: o padrão Fortran é o índice iniciar em "1".

Como definir um vetor?

Fazendo com que o índice não comece em 1:

```
REAL(KIND=8)::  a(-2:2)
```

Como definir um vetor?

Fazendo com que o índice não comece em 1:

```
REAL(KIND=8)::  a(-2:2)
```

```
a(-2) a(-1) a(0) a(1) a(2)
```

Como definir um vetor?

Fazendo com que o índice não comece em 1:

```
REAL(KIND=8)::  a(-2:2)
```

```
a(-2) a(-1) a(0) a(1) a(2)
```

OBS: dois pontos (“:”) separam os valores inicial e final de um índice.

Como declarar?

```
REAL(KIND=8),DIMENSION(5):: a,b,c
```

Como declarar?

```
REAL(KIND=8),DIMENSION(5):: a,b,c

REAL(KIND=8):: a(5),b(5),c(5)
```

Como declarar?

```
REAL(KIND=8),DIMENSION(5):: a,b,c

REAL(KIND=8):: a(5),b(5),c(5)

REAL(KIND=8),DIMENSION(5):: a,b(3),c
```

Como declarar?

```
REAL(KIND=8),DIMENSION(5):: a,b,c

REAL(KIND=8):: a(5),b(5),c(5)

REAL(KIND=8),DIMENSION(5):: a,b(3),c

REAL(KIND=8),DIMENSION(-1:4):: a
```

Como definir uma matriz?

```
REAL(KIND=8)::  a(3,4)
```

```
REAL(KIND=8)::  a(3,4)
```

O resultado dessa declaração será a criação de 12 espaços de memória denotados por:

Como definir uma matriz?

```
REAL(KIND=8):: a(3,4)
```

O resultado dessa declaração será a criação de 12 espaços de memória denotados por:

```
a(1,1) a(1,2) a(1,3) a(1,4)
a(2,1) a(2,2) a(2,3) a(2,4)
a(3,1) a(3,2) a(3,3) a(3,4)
```

```
REAL(KIND=8):: a(3,4)
```

O resultado dessa declaração será a criação de 12 espaços de memória denotados por:

```
a(1,1) a(1,2) a(1,3) a(1,4)
a(2,1) a(2,2) a(2,3) a(2,4)
a(3,1) a(3,2) a(3,3) a(3,4)
```

OBS: a vírgula (“,”) separa diferentes índices.

Como definir uma matriz?

Mudando o intervalo dos índices:

```
REAL(KIND=8)::  a(0:2,-1:2)
```

Como definir uma matriz?

Mudando o intervalo dos índices:

```
REAL(KIND=8)::  a(0:2,-1:2)

a(0,-1) a(0,0) a(0,1) a(1,2)
a(1,-1) a(1,0) a(1,1) a(1,2)
a(2,-1) a(2,0) a(2,1) a(1,2)
```

Diferentes Arranjos (Arrays)

- Um vetor pode ser visto como um arranjo de números A com apenas um índice: A_i;

Diferentes Arranjos (Arrays)

- Um vetor pode ser visto como um arranjo de números A com apenas um índice: A_i;
- Uma matriz pode ser vista como um arranjo de números A com dois índices: $A_{i,j}$;

Diferentes Arranjos (Arrays)

- Um vetor pode ser visto como um arranjo de números A com apenas um índice: A_i;
- Uma matriz pode ser vista como um arranjo de números A com dois índices: $A_{i,j}$;
- Um arranjo de números A com três índices ($A_{i,j,l}$) é declarado como:

```
REAL(KIND=8)::  a(2,3,5)
```

Diferentes Arranjos (Arrays)

- Um vetor pode ser visto como um arranjo de números A com apenas um índice: A_i;
- Uma matriz pode ser vista como um arranjo de números A com dois índices: $A_{i,j}$;
- Um arranjo de números A com três índices ($A_{i,j,l}$) é declarado como:

```
REAL(KIND=8)::  a(2,3,5)
```

```
a(1,1,1) a(1,2,1) a(1,3,1) a(2,1,1) a(2,2,1) a(2,3,1)
a(1,1,2) a(1,2,2) a(1,3,2) a(2,1,2) a(2,2,2) a(2,3,2)
a(1,1,3) a(1,2,3) a(1,3,3) a(2,1,3) a(2,2,3) a(2,3,3)
a(1,1,4) a(1,2,4) a(1,3,4) a(2,1,4) a(2,2,4) a(2,3,4)
a(1,1,5) a(1,2,5) a(1,3,5) a(2,1,5) a(2,2,5) a(2,3,5)
```

Escrevendo/Lendo dados de uma Array

```
WRITE(*,*) (a(2,i),i=2,7)
```

Escrevendo/Lendo dados de uma Array

```
WRITE(*,*) (a(2,i),i=2,7)

WRITE(*,*) ((a(i,j),i=2,7),j=1,3)
```

```
REAL(KIND=8)::  a(3),b(3),c(3)
```

Operações com Arrays

```
REAL(KIND=8)::  a(3),b(3),c(3)

  c(1)=a(1)+b(1)
```

Operações com Arrays

```
REAL(KIND=8)::  a(3),b(3),c(3)

  c(1)=a(1)+b(1)
  c(2)=a(2)+b(2)
```

Operações com Arrays

```
REAL(KIND=8)::  a(3),b(3),c(3)

  c(1)=a(1)+b(1)
  c(2)=a(2)+b(2)
  c(3)=a(3)+b(3)
```

Operações com Arrays

```
REAL(KIND=8)::  a(3),b(3),c(3)

  DO i=1,3
    c(i)=a(i)+b(i)
  END DO
```

Operações com Arrays

```
REAL(KIND=8)::  a(3),b(3),c(3)

  c=a+b
```

Operações com Arrays

```
REAL(KIND=8)::  a(3),b(3),c(3)

  c=a+b
```

OBS: Possível apenas a partir do FORTRAN 90.

Operações com Arrays

```
REAL(KIND=8)::  a(3),b(3),c(3)

  c=a+b
```

OBS: Possível apenas a partir do FORTRAN 90.

OBS: Não é permitido no FORTRAN 77.

Operações com Arrays

```
REAL(KIND=8)::  a(3),b(3),c(3)
  c=a+b
  c=a-b
  c=a*b
  c=EXP(a)
  c=SIN(a)
```

```
REAL(KIND=8)::a(3),b(3),c(5,3)
```

Operações com Arrays

```
REAL(KIND=8)::a(3),b(3),c(5,3)
  c(5,:)=a+b
  c(5,:)=a-b
  c(5,:)=a*b
```

```
REAL(KIND=8)::a(3,7),b(3),c(3)
```

Operações com Arrays

```
REAL(KIND=8)::a(3,7),b(3),c(3)
  c=a(:,7)+b
  c=a(:,7)-b
  c=a(:,7)*b
```

```
REAL(KIND=8)::a(7),b(3),c(3)
```

Operações com Arrays

```
REAL(KIND=8)::a(7),b(3),c(3)
  c=a(3:5)+b
  c=a(3:5)-b
  c=a(3:5)*b
```

Funções Intrínsecas com Arrays

```
MAXVAL(A)
MINVAL(A)
MINLOC(A)
MAXLOC(A)
SUM(A)
TRANSPOSE(A)
```

Arrays Alocatáveis

São matrizes, vetores e etc cujo tamanho pode ser determinado durante a execução do programa.

Arrays Alocatáveis

São matrizes, vetores e etc cujo tamanho pode ser determinado durante a execução do programa.

Como declarar?

```
INTEGER,ALLOCATABLE::  v(:),A(:,:),B(:,:,:)
```

Arrays Alocatáveis

São matrizes, vetores e etc cujo tamanho pode ser determinado durante a execução do programa.

Como declarar?

```
INTEGER,ALLOCATABLE::  v(:),A(:,:),B(:,:,:)
```

Como alocar?

```
ALLOCATE(A(10,20))
ALLOCATE(v(10),B(2,5,8))
```

Arrays Alocatáveis

Pode desalocar?

Arrays Alocatáveis

Pode desalocar? SIM!

Arrays Alocatáveis

Pode desalocar? SIM!

```
DEALLOCATE(A,v,B)
```

Arrays Alocatáveis

Pode desalocar? SIM!

```
DEALLOCATE(A,v,B)
```

Pode alocar de novo depois?

Arrays Alocatáveis

Pode desalocar? SIM!

```
DEALLOCATE(A,v,B)
```

Pode alocar de novo depois? SIM!

Arrays Alocatáveis

Pode desalocar? SIM!

```
DEALLOCATE(A,v,B)
```

Pode alocar de novo depois? SIM!

Regras de ouro:

- Não alocar o que já está alocado;
- Não desalocar o que não está alocado.

Arrays Alocatáveis

Pode desalocar? SIM!

```
DEALLOCATE(A,v,B)
```

Pode alocar de novo depois? SIM!

Regras de ouro:

- Não alocar o que já está alocado;
- Não desalocar o que não está alocado.

Dá pra verificar antes?

Arrays Alocatáveis

Pode desalocar? SIM!

```
DEALLOCATE(A,v,B)
```

Pode alocar de novo depois? SIM!

Regras de ouro:

- Não alocar o que já está alocado;
- Não desalocar o que não está alocado.

Dá pra verificar antes?

```
logical_var=ALLOCATED(A)
```

Arrays Alocatáveis

Pode desalocar? SIM!

```
DEALLOCATE(A,v,B)
```

Pode alocar de novo depois? SIM!

Regras de ouro:

- Não alocar o que já está alocado;
- Não desalocar o que não está alocado.

Dá pra verificar antes?

```
logical_var=ALLOCATED(A)
```

Mas é melhor saber de antemão se a array está alocada ou não.

Como testar uma (des)alocação

Como testar uma (des)alocação

```
REAL(KIND=8)::  a(:)
INTEGER::  ios

ALLOCATE(a(5),IOSTAT=ios)
IF(ios.ne.0)THEN
  WRITE(*,*) 'Falha na alocação de a.'
  STOP
END IF

DEALLOCATE(a,IOSTAT=ios)
IF(ios.ne.0)THEN
  WRITE(*,*) 'Falha na desalocação de a.'
  STOP
END IF
```

Mãos à Obra

Ellis book, exercises:

- 7.1
- 7.2
- 7.5
- 7.7
- 7.8
- 7.14